

Résumé

Assigame est une marketplace locale développée dans le cadre d'un projet d'études à l'ESGIS. La plateforme permet à des vendeurs de publier des annonces produits et à des visiteurs de consulter un catalogue public filtrable. Un espace administrateur assure la modération des inscriptions vendeur et des publications avant leur mise en ligne. L'application repose sur une architecture monolithique Spring Boot (Java 17) avec API REST sécurisée par JWT, un frontend HTML/CSS/JavaScript et une base PostgreSQL. Ce document présente les exigences, la conception UML, l'architecture technique, les interfaces clés et le bilan du projet.

Mots-clés : Marketplace, Spring Boot, JWT, modération, PostgreSQL, API REST, vendeur, catalogue

Note : générer la table des matières dans Word via Références → Table des matières → Mettre à jour.

Partie I — Cahier des charges

1. Introduction et contexte

Le commerce local nécessite des outils numériques accessibles. Assigame est une marketplace permettant aux vendeurs de publier des annonces et aux visiteurs de consulter un catalogue public. Un administrateur valide les inscriptions et les publications avant mise en ligne.

L'application est monolithique : Spring Boot sert l'API REST et les pages HTML statiques. L'authentification repose sur JWT ; les données sont stockées dans PostgreSQL.

- Permettre aux vendeurs de publier et gérer des annonces produits.
- Offrir un catalogue public filtrable sans compte visiteur.
- Garantir la qualité via modération administrateur.

2. Analyse du besoin

Problématique : comment permettre à des vendeurs locaux de publier des annonces tout en contrôlant la qualité des contenus ? La solution retenue impose une validation administrateur avant toute mise en ligne d'un compte vendeur ou d'un produit.

Périmètre inclus : catalogue public, inscription et authentification vendeur, espace vendeur (dashboard, publication, gestion), back-office admin (modération, catégories, offres). Hors périmètre : panier d'achat, paiement en ligne réel, compte client acheteur et messagerie interne. L'écran de paiement d'abonnement vendeur est une simulation UI.

3. Acteurs du système

Acteur	Description
Visiteur	Consulte le catalogue et les fiches produit

	sans compte ; contact vendeur par téléphone.
Vendeur	Publie et gère ses annonces selon son offre (Particulier, Professionnel, Partenaire Vip).
Administrateur	Modère inscriptions et produits ; gère catégories et types vendeur.

4. Exigences fonctionnelles et non fonctionnelles

ID	Exigence fonctionnelle
EF-01	Catalogue public : liste produits actifs, filtres, fiches détaillées, produits similaires
EF-02	Inscription vendeur : formulaire, choix d'offre, compte EN_ATTENTE jusqu'à validation
EF-03	Authentification vendeur par email/mot de passe (JWT)
EF-04	Espace vendeur : dashboard, publication (4-6 images), modification, suppression
EF-05	Quotas produits actifs : 5 / 15 / illimité selon l'offre
EF-06	Admin : connexion dédiée, dashboard KPI, modération vendeurs et produits
EF-07	Admin : gestion catégories (CRUD) et types vendeur (nom, description)

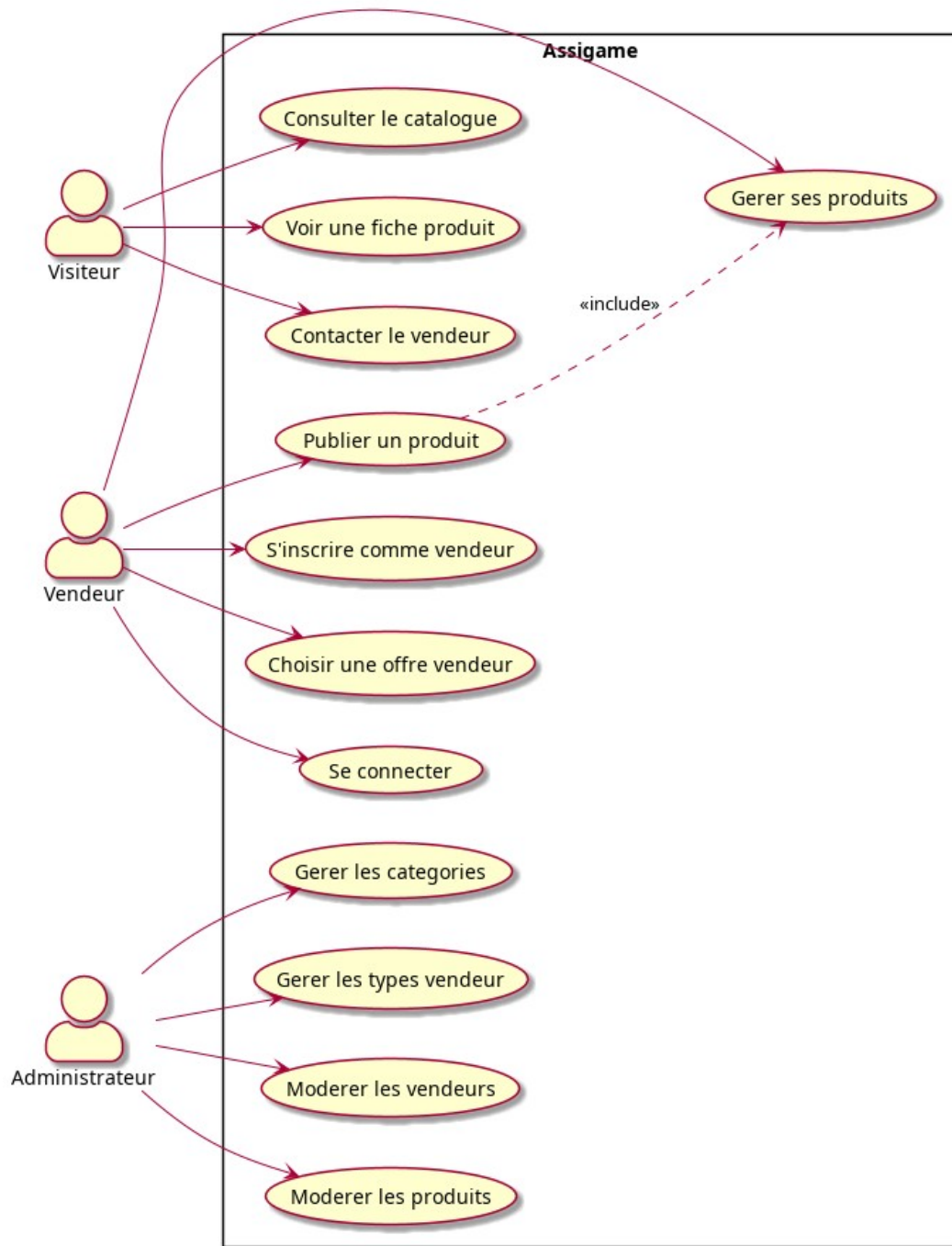
ID	Exigence non fonctionnelle
ENF-01	Authentification JWT stateless ; sessions admin et vendeur séparées
ENF-02	Mots de passe hashés BCrypt
ENF-03	API REST JSON ; persistance PostgreSQL via JPA
ENF-04	Upload images local (5 Mo max) ; interface HTML/CSS/JS responsive

- Contraintes : Java 17, Spring Boot 4, PostgreSQL, frontend sans framework.
- Risques : secret JWT en dev, stockage local des images, paiement abonnement simulé.

5. Cas d'utilisation

ID	Acteur	Cas d'utilisation	Résultat
UC-01	Visiteur	Consulter le catalogue	Produits ACTIF affichés
UC-02	Visiteur	Voir fiche produit / contacter vendeur	Détail et téléphone affichés
UC-03	Vendeur	S'inscrire et choisir une offre	Compte EN_ATTENTE créé
UC-04	Vendeur	Se connecter et gérer ses produits	JWT ; liste et CRUD produits
UC-05	Vendeur	Publier un produit	Produit EN_ATTENTE si quota OK
UC-06	Admin	Modérer vendeurs et produits	Statuts ACTIF ou REFUSE
UC-07	Admin	Gérer catégories et offres vendeur	Catalogue et offres à jour

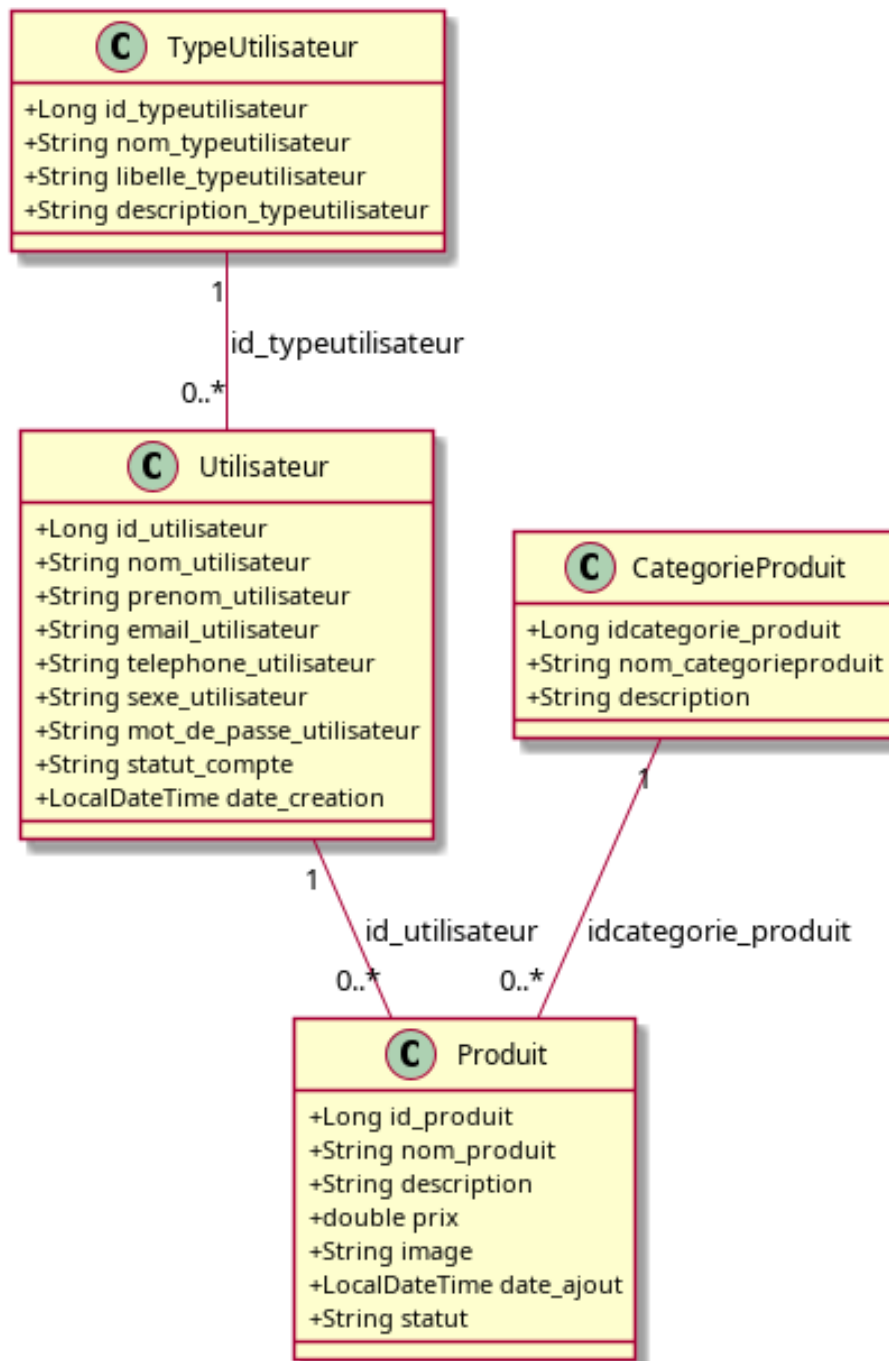
Figure F1 — Diagramme de cas d'utilisation



6. Diagramme de classes

Quatre entités JPA : TypeUtilisateur, Utilisateur, CategorieProduit, Produit. Relations : TypeUtilisateur 1→* Utilisateur 1→* Produit ; CategorieProduit 1→* Produit. Statuts : ACTIF, EN_ATTENTE, REFUSE, SUSPENDU.

Figure F2 — Diagramme de classes

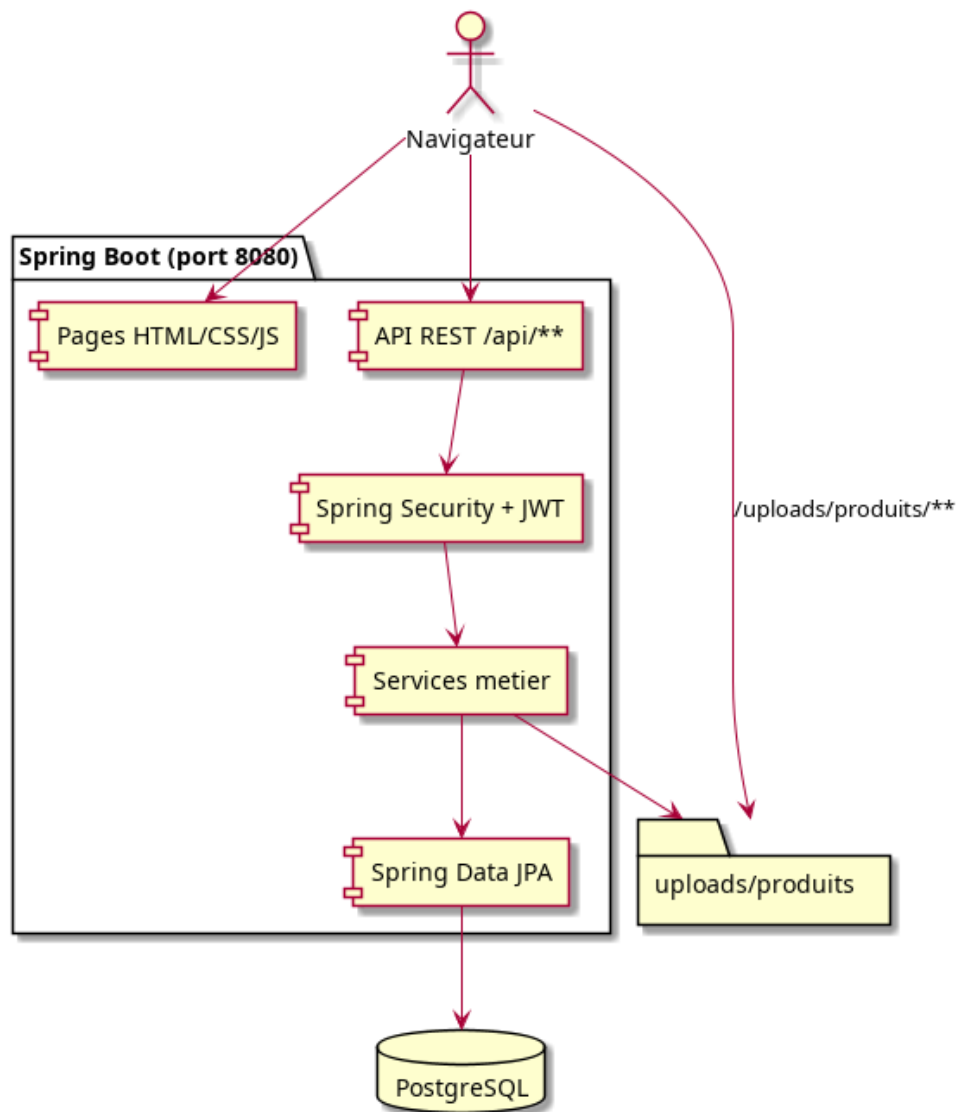


Partie II — Conception et réalisation

7. Architecture du système

Le navigateur accède aux pages statiques et à l'API REST. Spring Security valide les JWT. Les services métier utilisent JPA pour PostgreSQL ; les images sont dans uploads/produits/.

Figure F3 — Architecture



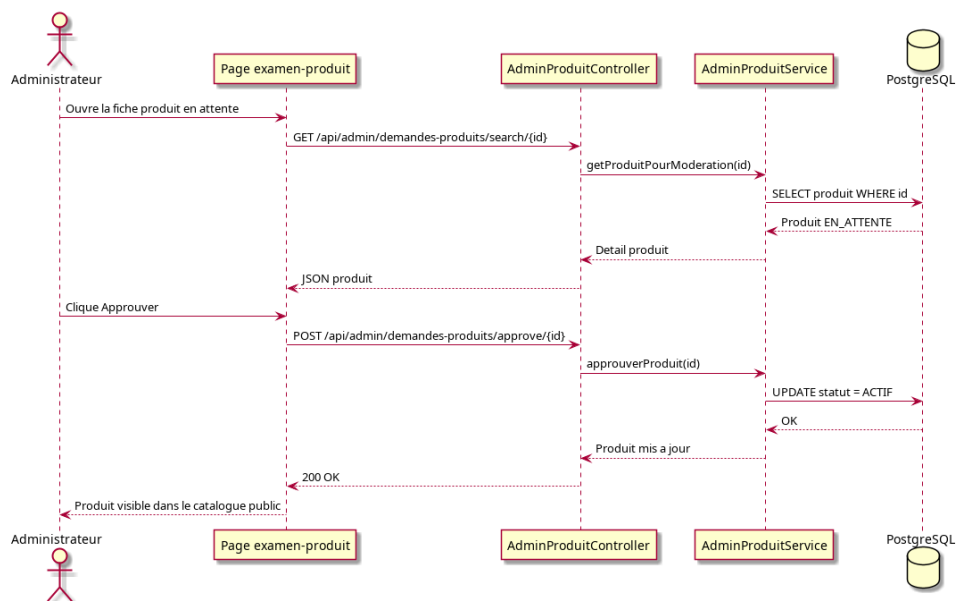
Couche	Technologie
Backend	Java 17, Spring Boot 4.0.6
Base de données	PostgreSQL, Spring Data JPA
Sécurité	Spring Security, JWT (jjwt)
Frontend	HTML5, CSS3, JavaScript vanilla
Build	Maven

8. Règles métier, sécurité et API

- 4 à 6 images obligatoires ; description max 2000 caractères.
- Quotas : Particulier 5, Professionnel 15, Partenaire Vip illimité (produits actifs).
- Modification vendeur → produit EN_ATTENTE ; seuls les ACTIF sont publics.
- JWT dans Authorization: Bearer ; tokens admin et vendeur séparés (localStorage).
- BCrypt pour les mots de passe ; vendeur ne modifie que ses propres produits.

Endpoint	Accès	Description
POST /api/auth/register	Public	Inscription vendeur
POST /api/auth/login	Public	Connexion → JWT
GET /api/produits/list	Public	Catalogue produits actifs
POST /api/produits/create	Vendeur	Créer un produit
GET /api/admin/demandes-vendeur/list	Admin	Demandes vendeur en attente
POST /api/admin/demandes-vendeur/approve/{id}	Admin	Approuver vendeur
GET /api/admin/demandes-produits/list	Admin	Produits en attente
POST /api/admin/demandes-produits/approve/{id}	Admin	Approuver produit

Figure F4 — Séquence : modération d'un produit



9. Interfaces utilisateur

Captures d'écran des principales interfaces de la plateforme Assigame.

Figure F5 — Catalogue public

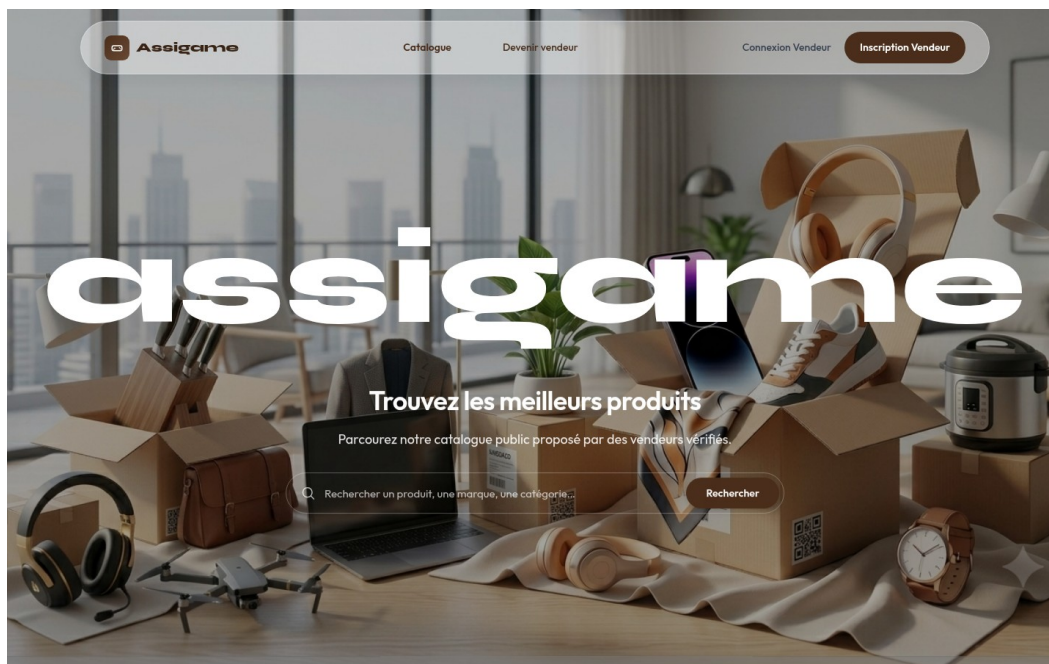


Figure F6 — Fiche produit

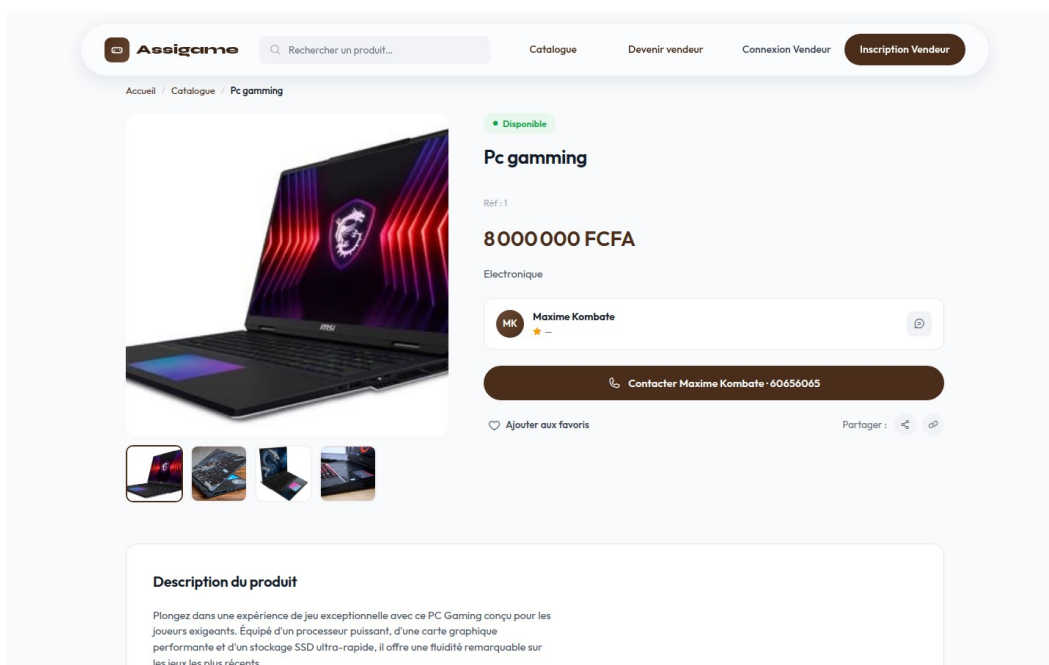


Figure F7 — Inscription vendeur

Aessigame

Catalogue

Devenir vendeur

Connexion Vendeur

Inscription Vendeur

Étape 1 sur 3 – Création du compte

Devenez Vendeur Partenaire

Complétez vos informations pour commencer votre inscription.

Informations personnelles

Conformes à l'API POST /api/auth/register.

Prénom

Votre prénom

Nom

Votre nom

Email

voire@email.com

Téléphone

00 00 00 00 00

Sexe

Homme

Femme

Sécurité du compte

Mot de passe

Confirmez

Espace vendeur et administration :

Figure F8 — Connexion vendeur

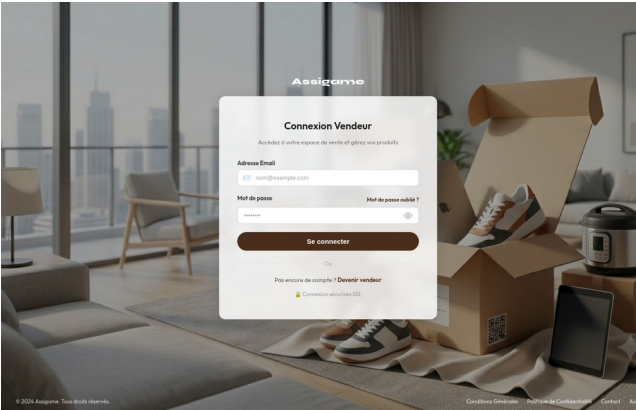


Figure F9 — Dashboard administrateur

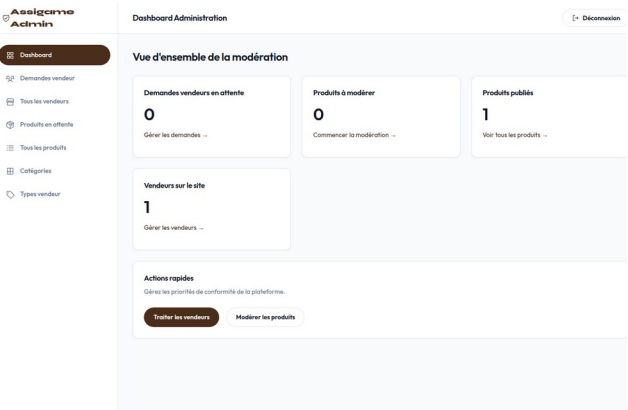


Figure F10 — Modération vendeurs

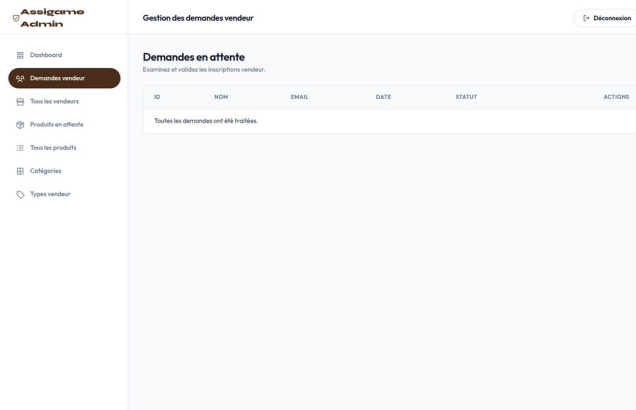


Figure F11 — Produits en attente

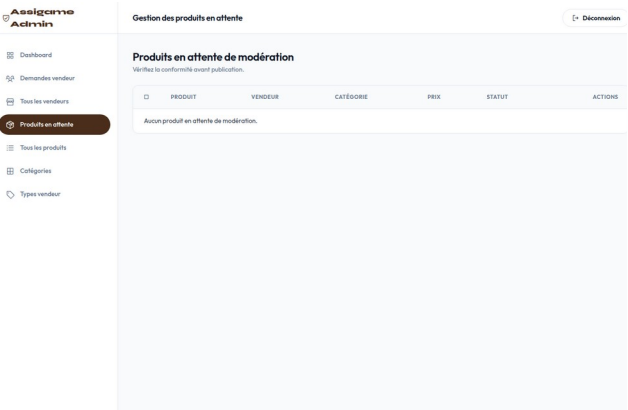


Figure F12 — Gestion des catégories

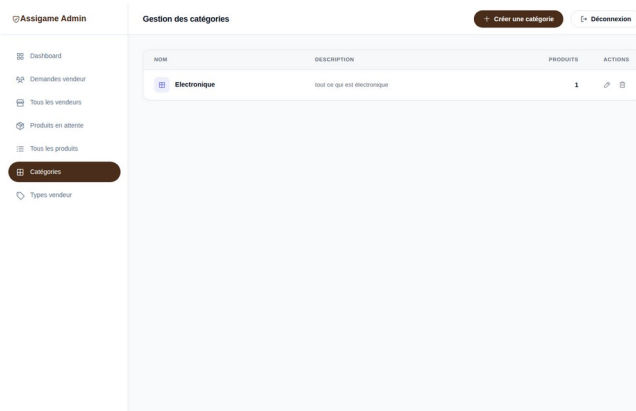


Figure F13 — Types vendeur

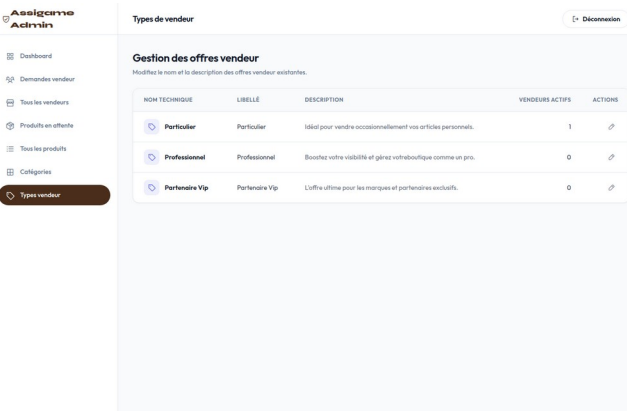
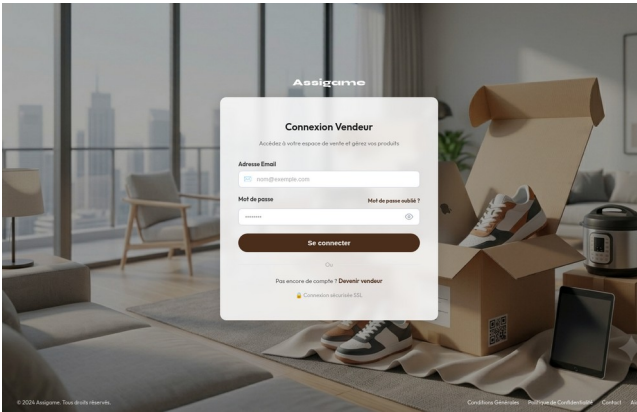


Figure F14 — Publication produit



Partie III — Validation et conclusion

10. Scénarios de test

ID	Scénario	Résultat attendu
T-01	Visiteur consulte le catalogue	Produits ACTIF affichés
T-02	Vendeur s'inscrit	Compte EN_ATTENTE
T-03	Admin approuve vendeur et produit	Compte et produit ACTIF
T-04	Quota Particulier dépassé	Erreur métier
T-05	Connexion JWT invalide	Accès API refusé

11. Difficultés rencontrées

- Erreurs 403 admin : résolu en réordonnant les règles /api/admin/** dans SecurityConfig.
- Comptage catégories : calcul dynamique via l'API produits au lieu d'un tiret en dur.
- Uploads : validation 4-6 images et stockage CSV côté service métier.
- Sessions : séparation assigne_admin_token et assigne_vendor_token.

12. Bilan, perspectives et conclusion

- Bilan : catalogue, parcours vendeur, modération et gestion catalogue livrés.
- Perspectives : paiement Stripe/Mobile Money, compte client, emails, déploiement cloud.

Assigame démontre la conception d'une marketplace complète : backend Java, modélisation relationnelle, sécurité JWT et intégration front statique. La modération centralisée constitue le cœur fonctionnel du projet.

Annexes

A. Arborescence du projet

```
assigame/  
├─ src/main/java/.../controller, service, entity, repository, security, config  
├─ src/main/resources/static/ (HTML, CSS, JS – admin/, vendeur/)  
├─ src/main/resources/application.properties  
└─ uploads/produits/ (images uploadées)
```

B. Installation et comptes test

- Prérequis : Java 17, Maven 3.8+, PostgreSQL 14+.
- Créer la base : CREATE DATABASE basespring;
- Lancer : mvn spring-boot:run → http://localhost:8080
- Admin : admin@assigame.com / Admin1234

C. Bibliographie

- Spring Boot — <https://spring.io/projects/spring-boot>
- Spring Security — <https://docs.spring.io/spring-security/reference/>
- PostgreSQL — <https://www.postgresql.org/docs/>
- JWT — <https://jwt.io/>